FinOpsBench: A Benchmark for Evaluating Agentic LLMs on Financial Analysis Tasks with Tool Use

Anonymous ACL submission

Abstract

We introduce FinOpsBench, a comprehensive benchmark for evaluating the capabilities of large language models (LLMs) in agentic financial analysis tasks that require tool use. The benchmark is released in two versions: (1) FinOpsBench-v1, a synthetic set of 5,979 natural-language financial analysis tasks that an agent must solve by issuing structured queries to a tool-callable data environment, with quality control by a panel of LLM judges, and (2) FinOpsBench-v2, a set of 1,108 multi-hop reasoning tasks grounded in real financial documents and embedded in custom tool environments. Both subsets require models to retrieve information exclusively through tool calls rather than relying on in-context information, testing genuine agentic tool use and reasoning capabilities. We evaluate a range of frontier and open-source models, finding that even the strongest model achieves only 68.9% on FinOpsBench-v1and 69.6% on FinOpsBench-v2, while open-source baselines fall as low as 21.9% and 16.3% respectively, demonstrating significant room for improvement in agentic financial reasoning. We release the full benchmark, evaluation code, and model traces to facilitate future research.

1 Introduction

Large language models (LLMs) are increasingly deployed as autonomous agents capable of using tools to accomplish complex tasks (Schick et al., 2023; Qin et al., 2023b). In the financial domain, such agents can assist with financial analysis, compliance checking, and decision support by issuing tool calls against structured business data and synthesizing the retrieved evidence into actionable answers.However, evaluating the agentic capabilities of LLMs presents significant challenges. Existing financial QA benchmarks like FinQA (Chen et al., 2021) and TAT-QA (Zhu et al., 2021) primarily test reading comprehension and numerical reasoning over provided contexts, rather than genuine tool-use and multi-step reasoning. We introduce FinOpsBench, a benchmark specifically designed to evaluate agentic LLMs on financial analysis tasks requiring tool use. We argue an agentic financial benchmark should at once provide diagnostic control over a large pool of executable tasks (so that failures can be traced to a specific planning or tool-use mistake rather than to evaluator noise) and realism on human-authored financial questions (so that accuracies remain comparable with the financial-QA literature). These two goals pull in opposite directions, so FinOpsBenchis released as two complementary parts - each targeting one of these goals and both evaluated under the same agent environment:

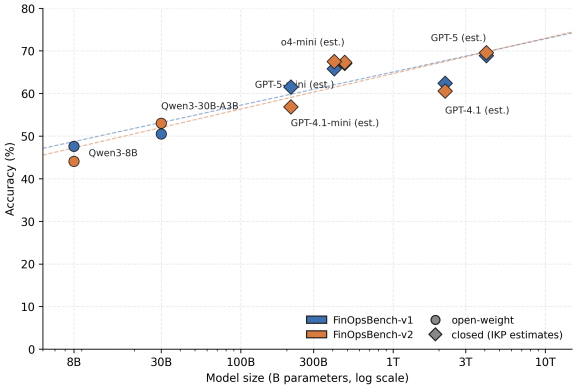

Figure 1: Accuracy on FinOpsBenchgrows log-linearly with the (estimated) parameter count of the agent's base model. Closed-source sizes are taken from the IKP estimates (Li, 2026)

.

- **FinOpsBench-v1**: 5,979 examples; each pairs a natural-language financial question with a small, freshly generated database that the agent must query through a single structured-data tool to reach the answer. Examples are quality-controlled by a panel of LLM judges.
- **FinOpsBench-v2**: This benchmark is pro-

duced by building agentic environments around questions and tables taken from FinQA, and consists of 1,108 samples. Each sample bundles a financial question, a tool environment with diverse custom Python tools, distractor tools and rows, and a gold answer. The agent must perform multi-hop reasoning across several tool calls.

Both benchmarks share the same design principles. First, answers cannot be extracted from the prompt alone, so agents must use tools to retrieve information. Second, examples require multi-step reasoning with intermediate tool calls. Third, distractor information is present to test the agent's ability to filter relevant from irrelevant data. A related concern with benchmarks is data contamination, where test examples may have been seen during model training. Where part of our benchmark builds on a prior dataset, each item is re-instantiated inside a bespoke tool environment with a redacted system prompt and a multi-step tool-call plan the model has not seen during training, so familiarity with the underlying source items does not translate into a direct answer pathway.

Our main contributions are:

- A novel benchmark for evaluating agentic LLMs on financial tasks requiring genuine tool use.
- A rigorous benchmark construction pipeline with quality control by a panel of LLM judges
- Comprehensive evaluation of a range of frontier and open-source LLMs used as the *base models* inside the agents, revealing significant gaps in agentic financial reasoning capabilities.
- An open-source release[1] of the complete benchmark together with the evaluation framework and all model traces. We treat this release as a contribution in its own right, supporting reproducible research and direct reuse of our infrastructure for new agentic financial tasks.

As Figure 1 shows that the performance of different models on FinOpsBenchgrows log-linearly with the parameter count of the agent's base model. This is an expected sanity check that the benchmark's difficulty tracks model capability rather than a task-design artifact, and that performance gaps therefore reflect real differences in agentic competence. Parameter counts for closed-source models are taken from the IKP calibration of Li (2026).

[1]Code: https://anonymous.4open.science/r/FinOpsBench

2 Related Work

FinOpsBenchis situated at the intersection of three research areas: context-bound financial Question Answering (QA), general agentic tool-use evaluation, and domain-specific semantic parsing.

Context-Bound Financial QA. Foundational financial benchmarks like FinQA (Chen et al., 2021), ConvFinQA (Chen et al., 2022), TAT-QA (Zhu et al., 2021), and FinanceBench (Islam et al., 2023) measure numerical reasoning, tabular-textual integration, and reasoning over SEC filings within static, explicitly provided contexts. These established datasets are inherently non-agentic as they do not require dynamic interaction or external system calls; all necessary information is supplied in the prompt, testing reading comprehension rather than tool use or dynamic information retrieval. Our FinOpsBench-v2component transitions this evaluation to a dynamic environment where information must be actively retrieved through multi-hop tool calls, isolating and testing the core planning and execution logic.

Agentic and Tool-Use Benchmarks. General agent benchmarks, such as ToolBench (Qin et al., 2023a,b), MCP-Verse (Lei et al., 2025), AgentBench (Liu et al., 2023), and SWE-bench (Jimenez et al., 2024), establish the methodological standard for evaluating LLMs' planning abilities against a large breadth of tools and domains (web browsing, code execution, APIs at scale). While essential for assessing general tool augmentation, these benchmarks prioritize scale over domain depth, and none concentrate on financial analysis with structured-data tools. Financial analysis critically demands precision and specialized interpretation. By specializing the tool environment for finance, FinOpsBenchtests the *depth* of financial logic, revealing significant gaps where general planning skills fail to confer the necessary domain competence (16–70% accuracy on grounded financial reasoning tasks). Recent real-world financial benchmarks like FinGAIA (Zeng et al., 2025), the Finance Agent Benchmark (Bigeard et al., 2025), and FinAgentBench (Choi et al., 2025) prioritize realism by using live web data (SEC EDGAR) or SEC-filing retrieval, while FinOpsBenchis engineered for *diagnostic fidelity* using controlled, synthetic environments to attribute failures unequivocally to flawed planning logic, rather than external noise or

API variability. We further compare FinOpsBenchagainst FinAgentBench and the Finance Agent Benchmark numerically along the size–accuracy axis in Appendix A.

Tool Use over Structured Business Data A related line of work studies semantic parsing over structured data: cross-domain datasets such as Spider (Yu et al., 2018) and BIRD (Li et al., 2024) score a model on the fidelity of a single generated query string against a fixed schema. FinOpsBenchis deliberately *not* a semantic-parsing benchmark: the agent is never scored on a query string. Instead, FinOpsBench-v1evaluates the full agentic loop (tool selection, multi-turn interaction with the data tool, evidence aggregation across calls, and natural-language answer synthesis) over financially specialized environments. Two consequences follow that make the task qualitatively different from text-to-query parsing. First, success requires planning under partial observability: the schema and contents must be probed via tool calls rather than read from context. Second, financial domain semantics such as accounts-payable aging, variance attribution, and revenue recognition become first-class evaluation targets rather than incidental surface features. The structured-data tool is the means, not the end being measured.

Our approach leverages synthetic data generation and a rigorous quality control process, utilizing a panel of three LLM judges, to mitigate data contamination risks and ensure high integrity at scale, validating that observed low accuracy reflects genuine performance limitations.

Data Generation. Synthetic data generation has proven valuable for creating evaluation benchmarks at scale (Wang et al., 2022). Recent work has explored using LLMs to generate and verify synthetic datasets with quality control mechanisms (Prabhakar et al., 2025). Our approach utilizes these methods and adapts them to creating financial agentic datasets.

3 Benchmark Description

3.1 Overview

Following the rationale laid out in the introduction, FinOpsBenchcomprises two complementary parts (Table 1): FinOpsBench-v1materialises the diagnostic-control side of the split (a large, fully synthetic pool with executable ground truth), while FinOpsBench-v2materialises the realism side, anchored in human-authored questions over real financial documents.

Property	FinOpsBench-v1	FinOpsBench-v2
# Examples	5,979	1,108
Tool type	Single structured-data tool	Many custom Python tools
Multi-hop	≥ 1 hop	≥ 2 hops
Distractor data	Yes	Yes
Quality control	Panel	Execution-based

Table 1: Overview of the two FinOpsBenchversions.

3.2 FinOpsBench-v1

FinOpsBench-v1consists of financial analysis tasks that an agent must solve by issuing structured-data queries through a single tool. Each example is backed by an isolated SQLite store generated specifically for that example; the agent is never required to produce or score a query string, and only the final answer is evaluated. Each example contains *User role* (the persona making the query, e.g., CFO, auditor, financial analyst), *Query* (a natural language question about financial data), *Backing data schema* (a plausible company-data schema generated for this example), *Backing data* (realistic financial records including distractor rows), *Expected answer* (the ground-truth answer), and *Agent trace* (a reference solution demonstrating correct tool use).

FinOpsBench-v1spans diverse financial analysis categories including Accounts Payable (AP) analysis, Revenue recognition, Variance analysis, Data integrity and reconciliation, and Financial reporting. Representative queries spanning user roles and task categories are listed in Appendix D, and the empirical category mix of the full FinOpsBench-v1stream is reported in Appendix G.

3.3 FinOpsBench-v2

FinOpsBench-v2transforms FinQA samples into complete agentic environments. Each example includes *System prompt* (scenario description and available tools without table data), *Synthetic backing store* (an SQLite (Hipp, 2024) table set reconstructed from the original FinQA tables), *Core tools* (functions required to derive the answer), *Distractor tools* (irrelevant tools to test filtering ability), and *Correct plan* (reference multi-hop solution). Critically, the agent receives only the scenario description and tool definitions, not the underlying data table. This ensures agents must use tools to retrieve information rather than extracting answers

from context. Representative questions, together with a worked example showing the query, available tools, and a reference tool-call sequence, are given in Appendix E and Table 5.

4 Benchmark Construction

4.1 FinOpsBench-v1pipeline

FinOpsBench-v1is created through a 9-stage pipeline. Figure 2 (top) gives an end-to-end overview of the FinOpsBench-v1construction process; the individual stages are described in detail below.

Stage 1: Query Generation. We start with a few seed queries spanning financial analysis categories (in our experiment, we took 12 seed queries). These seeds are then expanded using an LLM, varying user roles and specific details. We utilize a prompt that takes one seed and generates 20 queries to increase diversity and run it multiple times to generate a large number of queries. We also use embedding-based similarity filtering (cosine threshold 0.9) to remove near-duplicates.

Stage 2: Schema Generation. For each query, an LLM generates a plausible data schema that is: (1) sufficient to answer the query, (2) realistic for a company's records, and (3) appropriately complex.

Stage 3: Data Generation. The LLM is then tasked with generating realistic data for each query and schema, including: (1) data necessary to answer the query, and (2) distractor rows to increase difficulty. The LLM is also required to provide the correct answer to the query consistent with the generated data.

Stage 4: Execution-Based Validation. All generated schema and data definitions are validated by execution against SQLite. Errors are iteratively fixed using LLM assistance (up to 5 attempts). We found that LLMs struggle with fixing small errors in (relatively) long generated programs, so we split the program into individual statements and fix them one by one.

Stage 5: Agent Trace Generation. An agent runs on each query. It is supplied with access to the example's backing data through the structured-data tool, with up to 6 rounds of interaction allowed. The full conversation trace is recorded.

At this point, we have a benchmark subset that contains a query, a data schema, backing data (records necessary to answer the query and distractor rows), intended answer to the query, and an agent trace demonstrating the solution to the query. However, there may be hallucinations, unnatural data, wrong tool calls, or wrong answers in the agent trace that need to be identified and fixed.

Stage 6: Panel Judgement. A panel of three LLM judges evaluates each example's data and agent trace on five criteria: **(1)** *Data naturalness*: Is the generated data realistic? **(2)** *Trace reasonableness*: Does the agent examine appropriate data? **(3)** *Trace soundness*: Is the reasoning logical? **(4)** *Grounding*: Is reasoning based only on tool outputs? **(5)** *Answer correctness*: Does the answer address the query?

An example passes only if it receives majority approval (2/3 judges) on *all* criteria. In that case, it proceeds to the next stage. Otherwise, it goes to the improvement loop (Stages 7 and 8). Note that the loop only runs once for each example; if the improved example fails the judgement again, it is rejected completely.

Stage 7: Feedback Reconciliation. The judges' criticisms are given to an LLM which is asked to aggregate them into actionable feedback. The LLM at this stage has access to most of the example's data, including the query, the data schema, the backing data, and the agent trace.

Stage 8: Feedback Application. The agent runs again with a prompt containing the query, the data schema, the previous agent trace, and the feedback produced in Stage 7. It still has access to the structured-data tool, and the conversation is limited to 6 rounds. The full conversation trace is recorded.

Stage 9: Second Judgement. The improved examples are then re-judged by the panel. If an example passes, it proceeds to the next stage. Otherwise, it is dropped from the benchmark.

Final Filtering. The final benchmark includes only examples that passed panel judgement. We run two filtering passes to ensure quality and complexity of the benchmark. The first pass uses an LLM to compare the final answer produced in the agent trace with the intended answer created in Stage 3. Additionally, we filter out examples where the agent did not actually invoke the structured-data tool (removing examples answerable from context alone).

4.2 FinOpsBench-v2pipeline

The FinOpsBench-v2pipeline transforms FinQA samples through 9 stages (Figure 2, bottom):

Stage 1: Initial Solution. An LLM creates a direct Python solution using the original table data, establishing the computational logic.

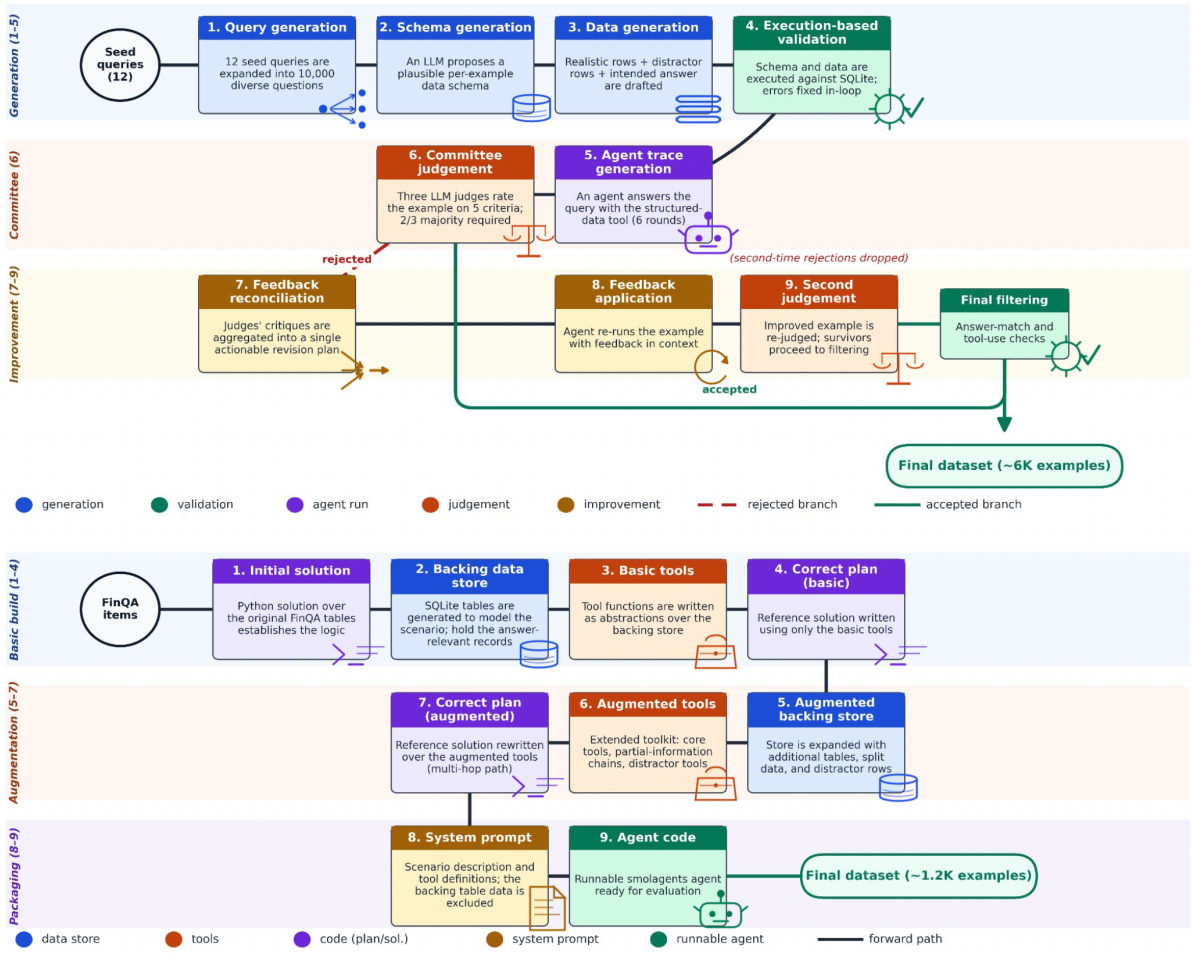

Figure 2: Construction pipelines for the two FinOpsBenchversions. **Top:** FinOpsBench-v1(9 stages with a panel-judgement gate and an improvement loop). **Bottom:** FinOpsBench-v2(9 strictly linear stages, each validated by execution).

Stage 2: Backing Data Store Generation. An SQLite-backed table set is generated to model the scenario, containing the records needed to answer the question.

Stage 3: Basic Tools. Tool functions are created as abstractions over the backing store, designed to enable multi-hop reasoning.

Stage 4: Correct Plan (Basic). A reference solution is written using the basic tools.

Stage 5: Augmented Backing Store. The backing store is expanded with additional tables, split data, and distractor rows to increase difficulty.

Stage 6: Augmented Tools. Extended tools are created including core tools for the correct solution, partial information tools requiring chaining, and distractor tools providing irrelevant data.

Stage 7: Correct Plan (Augmented). A reference solution using the augmented tools demonstrates the multi-hop solution path.

Stage 8: System Prompt. A system prompt is generated containing the scenario description and tool definitions, explicitly excluding table data and answer hints.

Stage 9: Agent Code. A runnable agent using the `smolagents` (Roucher et al., 2025) library is generated for evaluation.

Each stage is required to output a file (Python code in all stages except Stage 8 which outputs a system prompt with the task formulation). For each stage, the LLM receives in the context: (1) A long description of the whole process with all the stages described; (2) An indication of what stage is currently running; (3) All previous files' content from the previous stages, and (4) In Stage 9, an example agent code showing coding practices for achieving more stable results. After completing, each stage undergoes an execution-based validation. If the execution fails, the stage is rerun with a prompt that includes the error message and asks to fix the errors (up to 10 attempts).

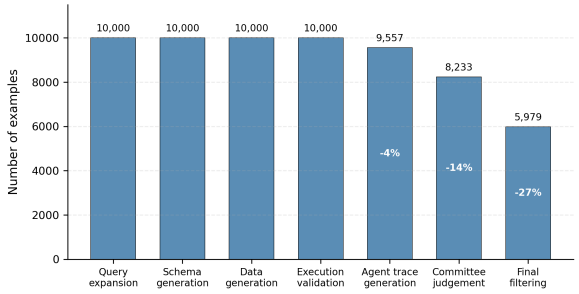

Figure 3: FinOpsBench-v1construction funnel: number of examples surviving each stage of the 9-stage pipeline plus the final filtering pass.

Stage	Number of Examples
Seed queries	12
1. Query expansion	10,000
2. Schema generation	10,000
3. Data generation	10,000
4. Execution-based validation	10,000
5. Agent trace generation	9,557
6. Panel judgement	
– passed	5,401
– to be improved	4,156
7. Feedback reconciliation	4,156
8. Feedback application	3,967
9. Panel judgement	2,832
Total passed so far	8,233
Filtering (post-pipeline)	**5,979**

Table 2: Number of examples in FinOpsBench-v1after each stage.

5 Experiments

5.1 Setup

We evaluate a diverse set of LLMs as the agent's base model. The frontier set is GPT-4.1, GPT-4.1-mini, GPT-5, GPT-5-mini, and o4-mini (OpenAI). The open-source set is Qwen3-8B and Qwen3-30B-A3B (Alibaba), and Llama-3.1-8B-Instruct (Meta). Frontier models are accessed through OpenAI's API; the three open-source models are served locally on a single NVIDIA H100 GPU. Our primary metric is *accuracy*. For FinOpsBench-v1, an LLM judge compares the agent's final answer against the expected output and outputs a yes/no verdict; we evaluate on a randomly selected test set of 729 examples for cost reasons. For FinOpsBench-v2, the numerical answers are compared against the correct plan output with a tolerance of one least significant digit, and we evaluate on the full set.

5.2 Construction statistics

We first describe how many examples survived each pipeline stage for the two versions.

FinOpsBench-v1. We constructed FinOpsBench-v1(5,979 examples) by starting with 12 seed queries and generating 10,000 diverse queries, then proceeding with all the remaining stages of the pipeline. The default LLM used in all stages is GPT-4.1-mini with the following exceptions: Stages 4 (execution-based validation), 7 (feedback reconciliation), and 8 (feedback application) are run with o4-mini; Stage 6 (panel judgement) is run with three different models: Claude Sonnet 4, o4-mini, and o3-mini, one per judge. See Table 2 for the number of examples after each stage.

Figure 3 visualises the same attrition: the pipeline holds steady through Stages 1–4 (deterministic generation against a fixed schema), then loses 4% of examples at the agent-trace stage, a further 14% at panel judgement, and 27% at the final filtering pass.

FinOpsBench-v2. We ran the data-example creation routine using the protocol described above for 1,247 examples drawn uniformly at random from the FinQA training split, resulting in 1,108 examples (the rest failed in at least one of the stages). Appendix F lists the per-example file layout.

5.3 Results

The final evaluation results for both versions of the benchmark are presented in Table 3.

Model	Acc. v1 (%)	Acc. v2 (%)
Frontier models		
GPT-5	**68.9**	**69.6**
GPT-5-mini	65.8	67.5
o4-mini	67.1	67.3
GPT-4.1	62.4	60.6
GPT-4.1-mini	61.5	56.9
Open-source models		
Qwen3-30B-A3B	**50.5**	**53.0**
Qwen3-8B	47.6	44.1
Llama-3.1-8B-Instruct	21.9	16.3

Table 3: Per-model accuracy on FinOpsBench-v1(v1) and FinOpsBench-v2(v2). Bold values mark the best score within each tier.

Findings. Having run experiments on both versions, we observe a consistent picture: overall model capability tracks benchmark accuracy, but the strongest models still leave substantial headroom. The log-linear dependence of accuracy

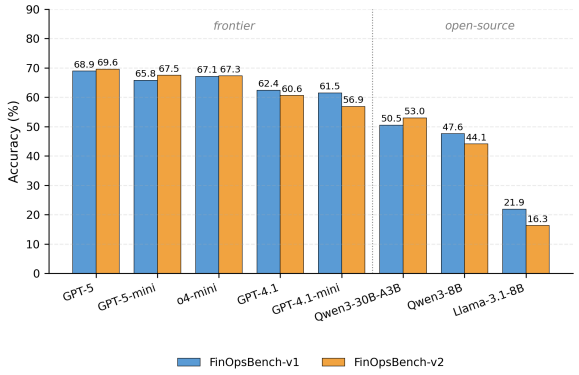

Figure 4: Per-model accuracy on FinOpsBench-v1and FinOpsBench-v2. Frontier models occupy the left half, open-source the right.

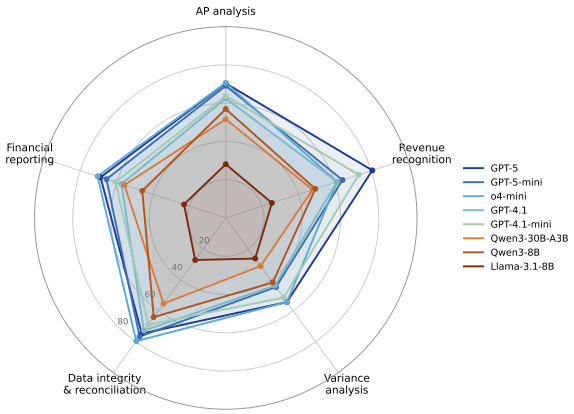

Figure 5: Per-category accuracy on FinOpsBench-v1for the eight evaluated models. Axes are the five categories described in Section 3.2; each query in the 729-example test set is assigned to one of these five categories. The category mix of the full FinOpsBench-v1stream is reported in Appendix G.

on the agent's base-model parameter count is already visible in Figure 1, and Table 3 provides the per-model numbers. On FinOpsBench-v1, even the best model (GPT-5) reaches only *68.9%*, indicating substantial room for improvement; modern and larger models such as GPT-5 outperform smaller, older ones such as Llama-3.1-8B; and the frontier/open-source gap is wide, with even Qwen3-30B-A3B scoring *11 pp* below the worst frontier model (GPT-4.1-mini). Figure 5 further shows that this ranking is stable across the four query categories carried by the seed set: every model's per-category polygon is approximately concentric, so the gap is not driven by any single financial subdomain.

On FinOpsBench-v2we observe essentially the same ranking. GPT-5 and o4-mini lead at *∼67–70%*, the GPT-4.1 variants follow, and the open-source models trail. Llama-3.1-8B again severely underperforms its size-matched newer sibling Qwen3-8B, suggesting that tool-use capability is sensitive to the training paradigm and not only to parameter count. Per-model accuracies track each other across the two versions (mean absolute difference *2.6 pp*); the side-by-side bar chart in Figure 4 and the $y = x$ scatter in Appendix B (Figure 9) make this agreement explicit , with the only material outliers being Qwen3-30B-A3B and Qwen3-8B, which both score noticeably lower on the FinQA-grounded version, where the multi-hop, multi-tool environment exposes more of the weaker models' failure modes.

Native tool calling vs. ReAct. While analyzing the traces of smaller models, we found it hard to understand what they were struggling with when calling tools, because there was no record of the reasoning behind the tool and argument choices. We therefore added a second experiment using the ReAct tool-calling protocol (Yao et al., 2022), which requires the model to output its reasoning before calling a tool. This experiment was performed on FinOpsBench-v1; Table 4 presents the results.

Model	Accuracy (%) Native	ReAct
Non-thinking Models		
GPT-4.1	62.4	63.8
GPT-4.1-mini	61.5	63.5
Llama-3.1-8B-Instruct	21.9	28.3
Thinking Models		
GPT-5	68.9	64.6
o4-mini	67.1	64.3
Qwen3-30B-A3B	50.5	46.2
Qwen3-8B	47.6	44.3

Table 4: Evaluation results on tool calling protocol. Underlined values are the best result for each model.

The split is clean and visible in Figure 6: every *non-thinking* model gains from switching to ReAct (a strict green delta on the right of the plot), while every *thinking* model loses (a strict red delta on the left). The gains shrink as the non-thinking model gets stronger, from *+6.4 pp* for Llama-3.1-8B-Instruct down to *+1.4 pp* for GPT-4.1, suggesting that ReAct's explicit reasoning scaffold matters most when the model lacks an internal chain-of-thought process and matters less, or even hurts, once the model already reasons before acting.

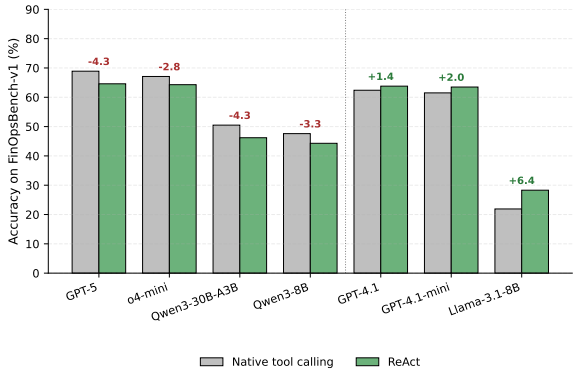

Figure 6: Native tool calling vs. ReAct on FinOpsBench-v1. Numbers above each pair give the ReAct minus Native delta in percentage points.

6 Discussion

Benchmarking Agentic Capabilities. Our results demonstrate that FinOpsBencheffectively discriminates between models' agentic capabilities. The significant gap from ceiling performance (around 69%) suggests current models have substantial room for improvement in financial tool use.

Open-Source Gap. The large gap between frontier and open-source models (up to 50+ percentage points) highlights an important direction for open-source development. Financial agentic tasks may require specialized training on tool-use trajectories. Concretely, closing this gap likely requires trajectory-level fine-tuning or reinforcement learning over realistic tool-use sequences (including tool selection, argument formation, and intermediate-result handling) rather than additional next-token pre-training on financial text alone.

Native Tool Calling vs. ReAct. ReAct allows the model to reason before choosing a tool to call. This has a large impact on performance for models that do not reason in their default setting, highlighting the improvement that test-time scaling can bring. For models that are already thinking, ReAct decreases performance. This may reflect that modern thinking models are well-optimized for structured function calling, while ReAct's verbose format introduces additional opportunities for error.

Quality Control. Our panel-based quality control ensures high-quality examples but introduces potential biases. The judges themselves are LLMs, which may share systematic blind spots with the evaluated models.

7 Conclusion

We introduced FinOpsBench, a benchmark for evaluating agentic LLMs on financial analysis tasks requiring tool use. The benchmark is released in two versions: FinOpsBench-v1(5,979 synthetically generated examples) and FinOpsBench-v2(1,108 FinQA-grounded examples). Evaluation of frontier and open-source LLMs used as the base models inside the agents reveals significant room for improvement, with best accuracies of 69%.

Our work provides a foundation for developing and evaluating financial AI agents. Future work may extend the benchmark to additional financial domains, incorporate real corporate data sources (with appropriate anonymization), and develop training approaches to improve agentic financial reasoning.

Ethics Statement

We plan to release FinOpsBenchunder the MIT License for research and educational use, in line with our goal of supporting external contributors and downstream reproducibility. FinOpsBench-v2is built on top of FinQA (Chen et al., 2021), which is itself distributed under the MIT License; our use of FinQA (building derivative tool environments around its public questions and tables) is consistent with its intended use as a public research benchmark, and we redistribute only derivative artifacts (synthetic backing stores, tool environments, reference plans, and agent traces) rather than the original FinQA tables verbatim.

FinOpsBenchcontains no personal or otherwise identifying data. FinQA is built on aggregated public corporate-finance disclosures from S&P 500 companies and contains no individual-person information; FinOpsBench-v1is generated end-to-end, and therefore contains no records traceable to real people or companies. No additional anonymisation step was required, and the benchmark and its accompanying artifacts are provided in English language.

During the preparation of our work, AI assistants were used for auxiliary tasks. A detailed description of this process is given in Appendix H.

Limitations

FinOpsBenchis scoped accordingly. Both versions currently instrument structured-data and Python-tool environments; other modalities such as live

web search, document retrieval, and multimodal financial reports lie outside this scope. These are deliberate scope choices for an initial release rather than fundamental limitations, and the construction methodology can in principle accommodate such extensions.

References

Anthropic. 2024. The claude 3 model family: Opus, sonnet, haiku. Technical report, Anthropic.

Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance Agent Benchmark: Benchmarking llms on real-world financial research tasks. *Preprint*, arXiv:2508.00828.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A dataset of numerical reasoning over financial data. *Proceedings of EMNLP 2021*.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. 2022. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. *Proceedings of EMNLP 2022*.

Chanyeol Choi, Jihoon Kwon, Alejandro Lopez-Lira, Chaewoon Kim, Minjae Kim, Juneha Hwang, Jaeseon Ha, Hojun Choi, Suyeol Yun, Yongjin Kim, and 1 others. 2025. FinAgentBench: A benchmark dataset for agentic retrieval in financial question answering. In *Proceedings of the 6th ACM International Conference on AI in Finance*, pages 632–637.

Richard D Hipp. 2024. SQLite.

Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. 2023. Financebench: A new benchmark for financial question answering. *Preprint*, arXiv:2311.11944.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Fei Lei, Yibo Yang, Wenxiu Sun, and Dahua Lin. 2025. MCPVerse: An expansive, real-world benchmark for agentic tool use. *Preprint*, arXiv:2508.16260.

Bojie Li. 2026. Incompressible knowledge probes: Estimating black-box LLM parameter counts via factual capacity. *arXiv preprint arXiv:2604.24827*.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, and 1 others. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2023. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv: 2308.03688*.

Akshara Prabhakar, Zuxin Liu, Ming Zhu, Jianguo Zhang, Tulika Awalgaonkar, Shiyu Wang, Zhiwei Liu, Haolin Chen, Thai Hoang, Juan Carlos Niebles, Shelby Heinecke, Weiran Yao, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. Apigen-mt: Agentic pipeline for multi-turn data generation via simulated agent-human interplay. *arXiv preprint arXiv:2504.03601*.

Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, Yi Ren Fung, Yusheng Su, Huadong Wang, Cheng Qian, Runchu Tian, Kunlun Zhu, Shihao Liang, Xingyu Shen, Bokai Xu, and 22 others. 2023a. Tool learning with foundation models. *Preprint*, arXiv:2304.08354.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023b. Toolllm: Facilitating large language models to master 16000+ real-world apis. *Preprint*, arXiv:2307.16789.

Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. 2025. 'smolagents': a smol library to build great agentic systems. `https://github.com/huggingface/smolagents`.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Preprint*, arXiv:2302.04761.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-Instruct: Aligning language model with self generated instructions.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium. Association for Computational Linguistics.

Lingfeng Zeng, Fangqi Lou, Zixuan Wang, Jiajie Xu, Jinyi Niu, Mengping Li, Yifan Dong, Qi Qi, Wei Zhang, Ziwei Yang, Jun Han, Ruilun Feng, Ruiqi Hu, Lejie Zhang, Zhengbo Feng, Yicheng Ren, Xin Guo, Zhaowei Liu, Dongpo Cheng, and 2 others. 2025. FinGAIA: A chinese benchmark for ai agents in real-world financial domain. *Preprint*, arXiv:2507.17186.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3277–3287, Online. Association for Computational Linguistics.

A Cross-Benchmark Comparison of Size–Accuracy Trends

Beyond the within-benchmark sanity check shown in Figure 1, where accuracy on FinOpsBenchgrows log-linearly with the agent's base-model size, we cross-check the size–accuracy relationship on two other recent financial-agent benchmarks. Parameter counts for closed-source models throughout this appendix come from the IKP calibration of Li (2026); we use the direct estimates from that paper's main table where available and invert the published log-linear calibration on the extended-table IKP scores otherwise.

FinAgentBench (Choi et al., 2025). On the document- and chunk-ranking tasks of FinAgentBench (Figure 7), the trend is the opposite of the one we observe on FinOpsBench: across the three evaluated proprietary models (Claude-Sonnet-4 at ≈237B, Claude-Opus-4 at ≈1.4T, and GPT-o3 at ≈3T), the *smallest* model wins or ties on every metric (nDCG@5, MAP@5, MRR@5) in both ranking tables. A monotonically negative slope on accuracy versus parameter count is a strong negative signal for benchmark validity. It suggests either that the task probes a narrow stylistic regime that the larger models are over-trained out of, or that smaller models in the same family have been exposed to closely related retrieval data during training. In short, performance on this benchmark appears to be governed more by model-family biases or potential data leakage than by raw model capability.

Finance Agent Benchmark (Bigeard et al., 2025). On the 22-model Finance Agent Benchmark leaderboard (Figure 8), accuracy does increase with parameter count, fitting a roughly +15.5 pp-per-decade log-linear trend. However, the residual variance around this trend is large enough that the fit is not, on its own, a reliable validity signal. First, at comparable sizes models differ by 20–40 pp; for example, Claude 3.7 Sonnet (≈676B) reaches 44% while GPT-4o (≈720B) reaches only 20%. Second, several large models sit far below the trend line: o1 at ≈3.5T scores only 21.4%, and LLaMA 4 Maverick at ≈400B scores only 3.1%, which the source paper attributes to tool-interface misuse. Third, and most strikingly, the same model run in two different reasoning modes can differ by 13 pp at fixed parameter count (Grok 3 Mini at 30.9% high-reasoning vs. 17.6% low-reasoning). The positive slope therefore conflates parametric capacity with reasoning paradigm and tool-use training; it does not isolate size as the dominant driver, in contrast to the cleaner relationship observed on FinOpsBench.

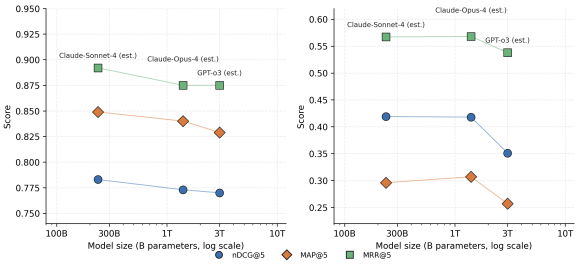

Figure 7: Size–accuracy on FinAgentBench (Choi et al., 2025), Tables 1 and 2. The smallest model wins or ties on every metric, an inverse trend that is inconsistent with what we observe on FinOpsBench.

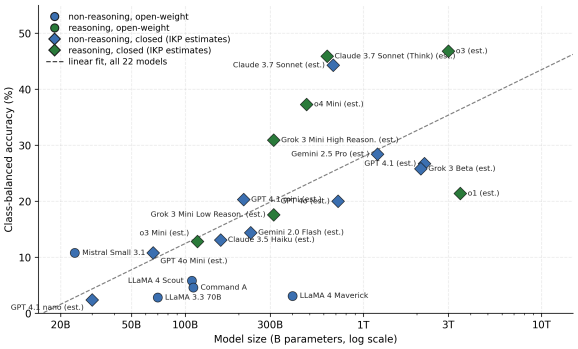

Figure 8: Size–accuracy on the Finance Agent Benchmark (Bigeard et al., 2025), Table 2. The positive log-linear slope (≈ 15.5 pp/decade) is consistent with the FinOpsBench picture, but the wide intra-size scatter (20–40 pp) and the same-model reasoning-mode variation indicate that size is not the dominant driver.

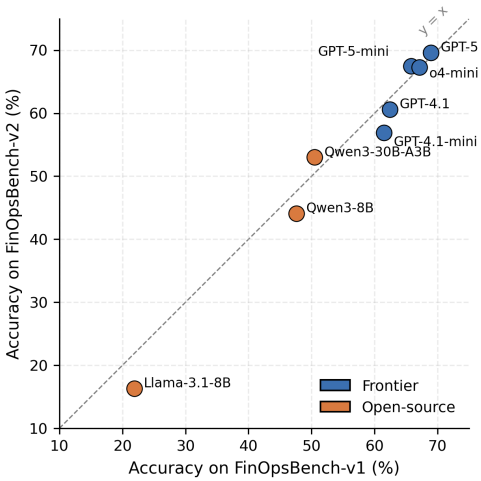

Figure 9: Per-model agreement between FinOpsBench-v1and FinOpsBench-v2accuracies. Models on the $y = x$ line score identically on both benchmarks; models below the line are weaker on FinOpsBench-v2.

B Per-model accuracies across the two FinOpsBenchversions

This appendix complements the side-by-side bar chart in Figure 4 with a per-model agreement view: Figure 9 plots the same data as a scatter against a $y = x$ reference line, separating models that score equally on the two versions from those that score lower on FinOpsBench-v2.

C Benchmark Statistics

This appendix collects descriptive statistics for both benchmark versions, organised by version below. Together the plots give a per-axis (text, schema, agent activity, tool surface) snapshot of how the two versions compare at the example level.

C.1 FinOpsBench-v1Statistics

The figures below summarise FinOpsBench-v1along three axes. Figures 10 and 11 cover the text side of the corpus (query length and the semantic spread of the queries in UMAP space). Figures 12 and 13 describe the size of the per-example backing stores in terms of table count and total row count. Figures 14 and 15 report the interaction depth of the reference agent traces (assistant turns and tool invocations per example), indicating how much agent activity each task induces.

C.2 FinOpsBench-v2Statistics

The figures below summarise FinOpsBench-v2. Figures 16–18 cover the answer-length distribution,

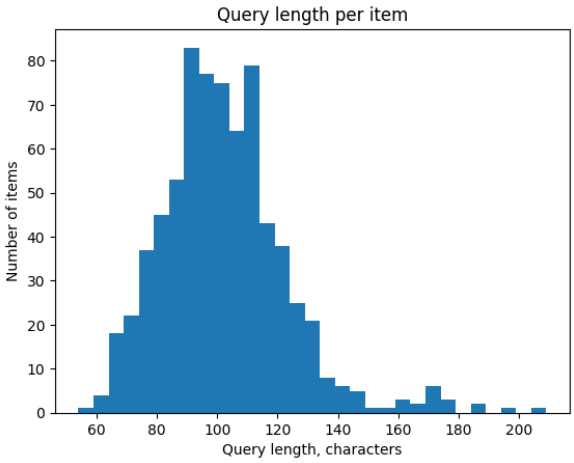

Figure 10: FinOpsBench-v1: Query length distribution.

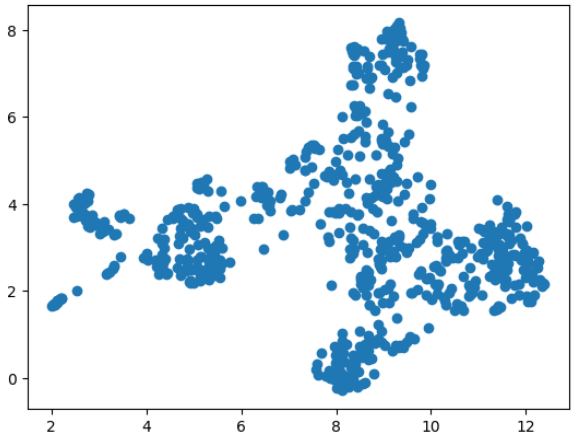

Figure 11: FinOpsBench-v1: UMAP of query embeddings.

the system-prompt-length distribution, and the per-example tool count. Figures 19 and 20 project the system prompts and the tool-name vocabularies into UMAP space to visualise topical and lexical coverage.

D Example Queries from FinOpsBench-v1

The following queries are drawn at random from the filtered FinOpsBench-v1test set. Each query is paired with the persona the LLM was instructed to assume when it generated the question; together they illustrate the breadth of user roles and the diversity of analysis categories covered by FinOpsBench-v1.

- *Internal Auditor*: Can you provide a list of transactions that were processed without obtaining necessary managerial approvals in the

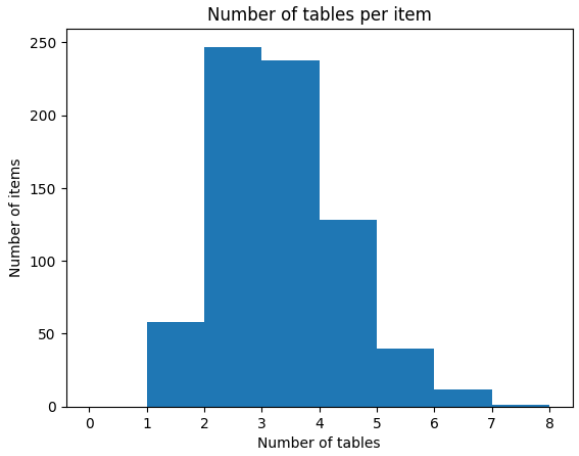

Figure 12: FinOpsBench-v1: Number of data tables per example.

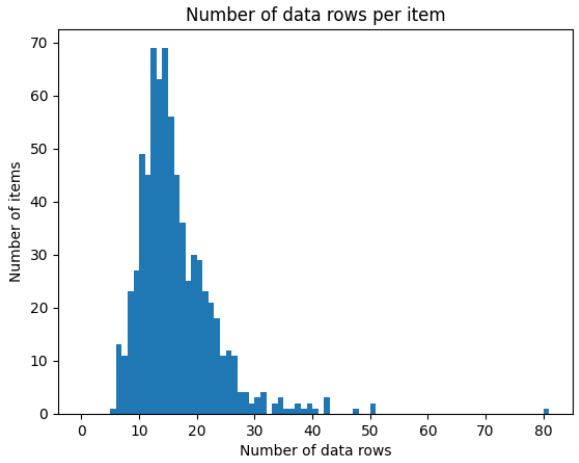

Figure 13: FinOpsBench-v1: Number of total data rows per example.

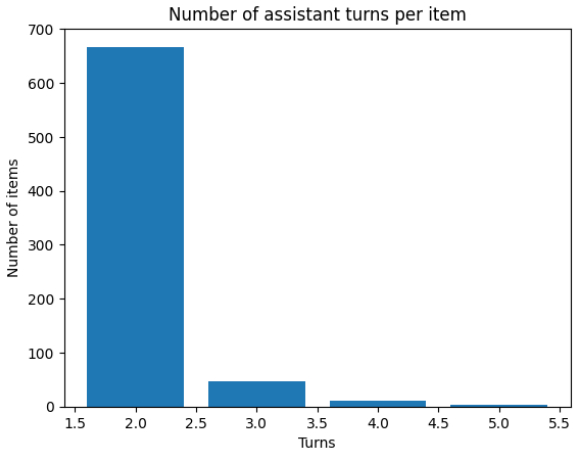

Figure 14: FinOpsBench-v1: Number of assistant turns per example.

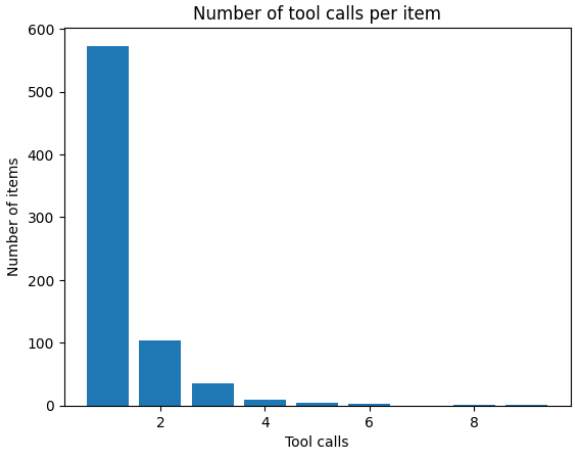

Figure 15: FinOpsBench-v1: Number of tool calls per example.

last quarter?

- *Tax Advisor*: Identify any vendor invoices that may impact VAT input tax claims for the current tax period.
- *Chief Financial Officer*: How do we ensure the reported accounts payable aging genuinely reflects outstanding vendor obligations without distortion?
- *Financial Controller*: What is the total outstanding balance on all overdue invoices by supplier category?
- *Procurement Officer*: Which suppliers have outstanding invoices that have not been approved for payment yet?
- *Finance Director*: What percentage of supplier invoices in the last financial year were paid late, grouped by supplier?
- *Risk Manager*: List suppliers with overdue invoices over 90 days that have credit limit warnings.
- *Internal Controls Manager*: Detect any payments made with override flags but missing authorization memos.

Across the full 5,979-example set we observe more than 60 distinct user roles (e.g. CFO, internal/external auditor, controller, procurement officer, treasury manager, tax advisor, compliance officer, risk manager), and queries span the accounts-payable, reconciliation, variance-analysis, revenue-recognition, and financial-reporting categories described in Section 3.2.

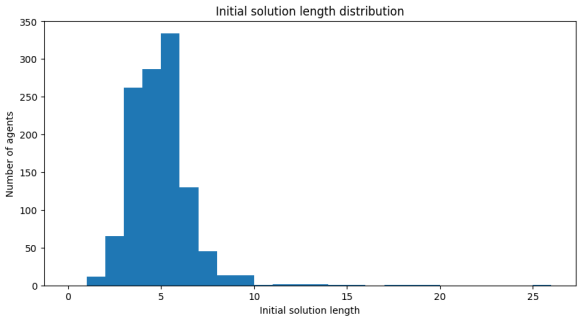

Figure 16: FinOpsBench-v2: Answer length distribution.

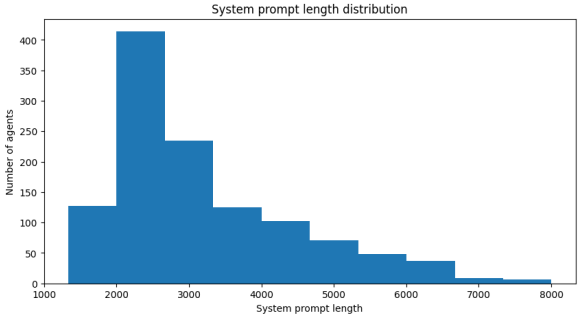

Figure 17: FinOpsBench-v2: System prompt length distribution.

E Example Questions from FinOpsBench-v2

The following questions are drawn at random from the filtered FinOpsBench-v2set. Each is a multi-hop numerical-reasoning question taken from a FinQA item; the agent must reach the answer by calling the example's bespoke Python tools rather than by reading the original FinQA table directly.

- What is the percent change in receivables from or (payables to) the Money Pool from 2001 to 2002?
- In 2018, what was the ratio of the qualified defined benefit pension plans' estimated payments for the period starting after 2024 compared with the amount for 2019?
- What percentage were North American consumer packaging net sales of consumer packaging sales in 2014?
- As of December 31, 2016, what was the percent of the total future minimum lease commitments for operating leases that was due in 2017?
- What was the change in percentage of the company's consolidated net sales from 2006 to 2008?

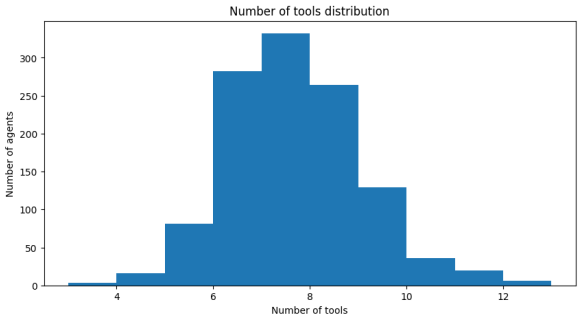

Figure 18: FinOpsBench-v2: Number of tools per example.

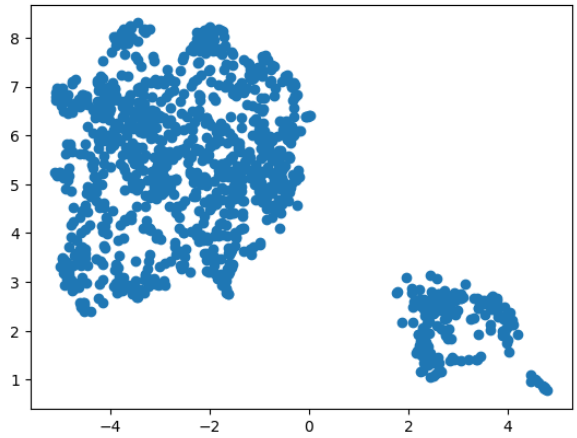

Figure 19: FinOpsBench-v2: UMAP of system prompt embeddings.

- For the year ended December 31, 2007, what was the ratio of the shares granted to the shares vested?
- If you held 1,000 shares on May 30, 2014, how much would you receive in dividends?
- What is the total, in millions, of current assets acquired?

Across the full 1,108-example set, every question is taken from FinQA and the surrounding tool environment is generated automatically (see Section 3.3). The questions span ratio computations, year-over-year changes, percentage shares of totals, and aggregate sums, mirroring the financial-reasoning surface of the original FinQA dataset and reinstantiating it as an agentic task.

Table 5 shows a fully worked example: the question text, the bespoke Python tools the agent has access to, and the reference tool-call sequence that solves the task.

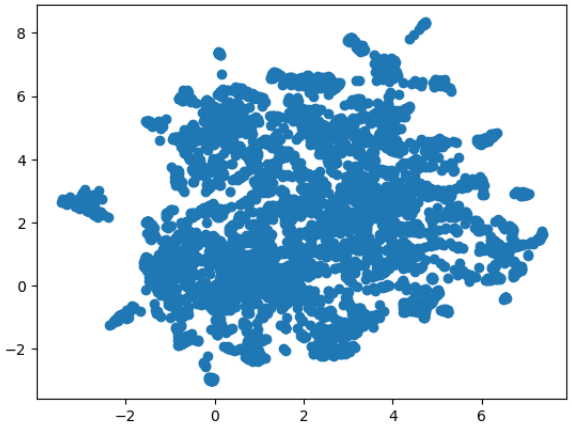

Figure 20: FinOpsBench-v2: UMAP of tool name embeddings.

Query (part of system prompt)

What is the percent change in receivables from or (payables to) the Money Pool from 2001 to 2002?

Available tools

```
list_available_years()
get_quarterly_money_pool_amount(year, q)
get_total_money_pool_amount_for_year(year)
compute_percent_change(old, new)
get_cash_flow_transactions(start, end, limit)
sum_cash_flow_for_year(year)
```

Correct tool-call sequence

```
def annual(y):
return sum(
get_quarterly_money_pool_amount(y, q)
for q in range(1, 5))
a01 = annual(2001)
a02 = annual(2002)
pct = compute_percent_change(a01, a02)
answer = f"{round(pct)}%"
```

Table 5: A worked FinOpsBench-v2example (excerpt): question, available tools, and reference tool-call sequence.

F Per-example file layout for FinOpsBench-v2

Each example in FinOpsBench-v2is shipped as a small directory of source files, listed below.

`initial_solution.txt`
Correct answer to the question.

`synthetic_db_generator_augmented.py`
Python code that creates the SQLite database.

`tools_augmented.py`
Tools required for the multi-step processing, plus distractor tools.

`tool_proxy.py`
Same tools, adapted for use with `smolagents`.

`correct_plan_augmented.py`
Intended sequence of tool calls that solves the task.

`agent_system_prompt.txt`
LLM prompt with context, tool descriptions, and the question.

`agent.py`
Agent code that runs a specified LLM with the system prompt and tools.

In addition, each example directory carries auxiliary intermediate files kept from the build process for reproducibility.

G Category distribution of FinOpsBench-v1

Table 6 reports the share of FinOpsBench-v1examples assigned to each of the five categories from Section 3.2. Categorisation is done automatically by nearest-anchor TF-IDF similarity over a short description sentence per category, applied to the full FinOpsBench-v1stream. The mix is dominated by accounts-payable queries (over half of the corpus), reflecting the seed-query composition; the remaining four categories are balanced between 7 and 15%.

Category	% of FinOpsBench-v1
Accounts Payable (AP) analysis	56.5
Variance analysis	14.8
Data integrity & reconciliation	13.5
Revenue recognition	8.2
Financial reporting	7.1

Table 6: Category share of the full FinOpsBench-v1stream (percentages, computed by nearest-anchor TF-IDF assignment). Rounded to one decimal; totals to 100.1% due to rounding.

H Prompts and Examples

Figure 22 shows the prompt template used for the panel LLM judge in FinOpsBench-v1.

Figure 21 shows an annotated example from FinOpsBench-v1illustrating the schema and data format.

I Use of AI Assistants

In the spirit of transparent reporting of AI-assisted authoring, we summarise how AI tools were used in the preparation of this paper. All

```
Query:
Could duplicated invoice payments be
causing discrepancies in our aged
payables report?

Schema:
CREATE TABLE Suppliers (
    supplier_id INTEGER PRIMARY KEY,
    supplier_name TEXT NOT NULL
);
CREATE TABLE Invoices (
    invoice_id INTEGER PRIMARY KEY,
    supplier_id INTEGER NOT NULL,
    invoice_date DATE NOT NULL,
    due_date DATE NOT NULL,
    invoice_amount NUMERIC NOT NULL,
    FOREIGN KEY (supplier_id)
    REFERENCES Suppliers(supplier_id)
);
CREATE TABLE Payments (
    payment_id INTEGER PRIMARY KEY,
    invoice_id INTEGER NOT NULL,
    payment_date DATE NOT NULL,
    payment_amount NUMERIC NOT NULL,
    FOREIGN KEY (invoice_id)
    REFERENCES Invoices(invoice_id)
);


Data:
INSERT INTO Payments VALUES
(5001, 1002, '2025-07-05', 500.00),
(5002, 1002, '2025-07-10', 500.00),
(5003, 1002, '2025-07-12', 500.00);
```

Figure 21: FinOpsBench-v1: Annotated example item showing query, schema, and data (excerpt).

core benchmark code, prompts, and configuration were written manually by the authors and reviewed end-to-end; the main body of the paper was likewise written manually. We used Claude (Anthropic, 2024) during the final preparation pass for two narrowly scoped tasks: (i) editorial revisions of phrasing and tightening of existing prose, and (ii) auxiliary code for collecting statistics about the benchmark and rendering some of the analysis figures. Every artifact produced by Claude was iteratively reviewed and accepted by the authors; no Claude output was incorporated without manual inspection.

```
# Task You are a senior QA engineer
working on AI agents for finance. You
have to judge the test case provided
and check the following rules:

- The test case's data should be
  natural: it should be similar to what
  data could a real company have, and
  how it would be stored in a database.
- The agent's trace should be
  reasonable: the agent should look at
  the data needed to answer the query.
- The agent's trace should be sound:
  the reasoning steps should be logical,
  should follow from the previous steps
  and lead to the final answer.
- The agent's reasoning and final answer
  should be grounded: the only data the
  agent has access to is what was
  returned from its tool calls. The
  reasoning should not be based on any
  other data, including the data in the
  database that was not queried with
  the tools.
- The agent's final answer should
  answer the user's query.

In case one of these requirements are
not met, you should provide a concise
actionable feedback so the author can
improve the test case.
# Test case:
## Query: {{ item.query }}
## Schema: {{ item.schema_sql }}
## Data: {{ item.data_sql }}
## Trace: {{item.agent_dialog|tojson(2)}}
Reply with JSON object like this:
{
   "data_is_natural": 1 | 0,
   "trace_is_reasonable": 1 | 0,
   "trace_is_sound": 1 | 0,
   "reasoning_is_grounded": 1 | 0,
   "answer_is_sound": 1 | 0,
   "feedback": "..."
}
```

Figure 22: FinOpsBench-v1: Prompt template for a panel member judge.